

COEN 448/6761 Software Testing and Validation Assignment 2 – Integration Testing and Test Coverage

[Individual or Group of 2 or Group of 3]

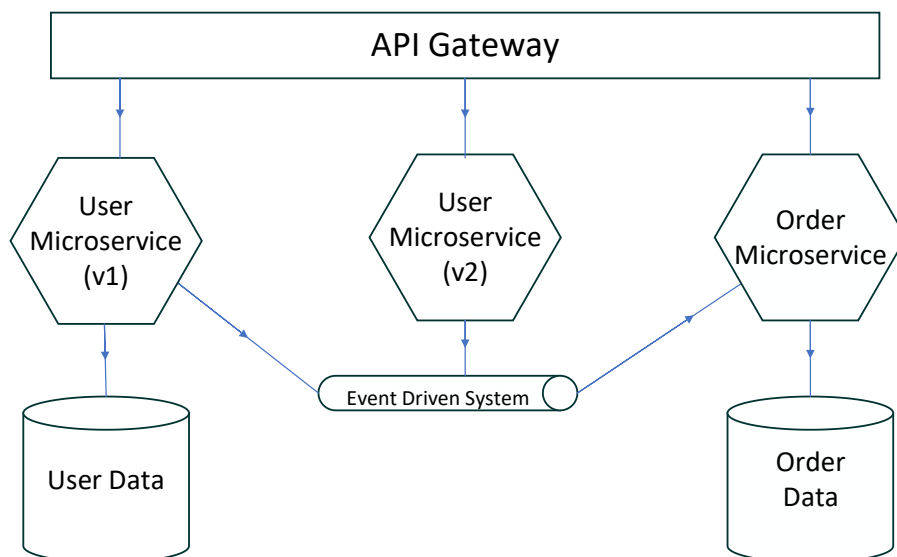
@copyright Yan Liu 2026-2027

This assignment is originally developed by Yan Liu @ Concordia University. This assignment is only for the course teaching and education purpose. Any distribution of this document to the Internet that involves any profit-making purpose is not given the consent from the author.

This assignment is to practice integrating testing. A Microservice system is provided and accessible from github. [GitHub - TheBarzani/aware-microservices: Scalable and Versionable Microservices Architecture Based on Modern Technologies](#). The overall architecture consists of microservices and CI/CD deployment. The detailed technical report is attached to the Moodle site.

Overview of the Microservice Architecture

The microservice architecture consists of services, data sets and frameworks. The overall architecture is depicted in the block diagram.



(1) The user database contains a user's information including at least the user account id, emails and delivery address. The order database contains a set of items ordered, the user emails and delivery address, unique order id and order status ("under

process", "shipping" and "delivered"). The email and delivery address are synchronized across two databases.

(2) The API gateway routes the HTTP requests to microservices. The **User Microservice** has a REST API to handle POST requests to create a new user; PUT requests to update the email address or delivery address. The **Order Microservice** has a REST API or OpenAPI to handle GET requests to retrieve orders with a state; POST requests to create a new order with items and email address and delivery address; PUT requests to update the status("under process", "shipping" and "delivered"); PUT requests to update emails or delivery address.

(3) When the email or delivery address is updated by a PUT request of the user microservice, the synchronization between the user database and the order database is triggered by an event in the event driven system.

(4) Introduce a new version (v2) of the user microservice. Some changes are refactored to the user microservice (v1).

(5) Develop the strangler pattern in the API gateway by routing **P** percentage of user requests to v1 and **(1-P)** percentage of user requests to v2. P can be specified in a configuration file associated with the API gateway. P should not be hardcoded in the API gateway.

(6) The user database and the order database can use MongoDB or Redis cloud service. The event driven system can use RabbitMQ.

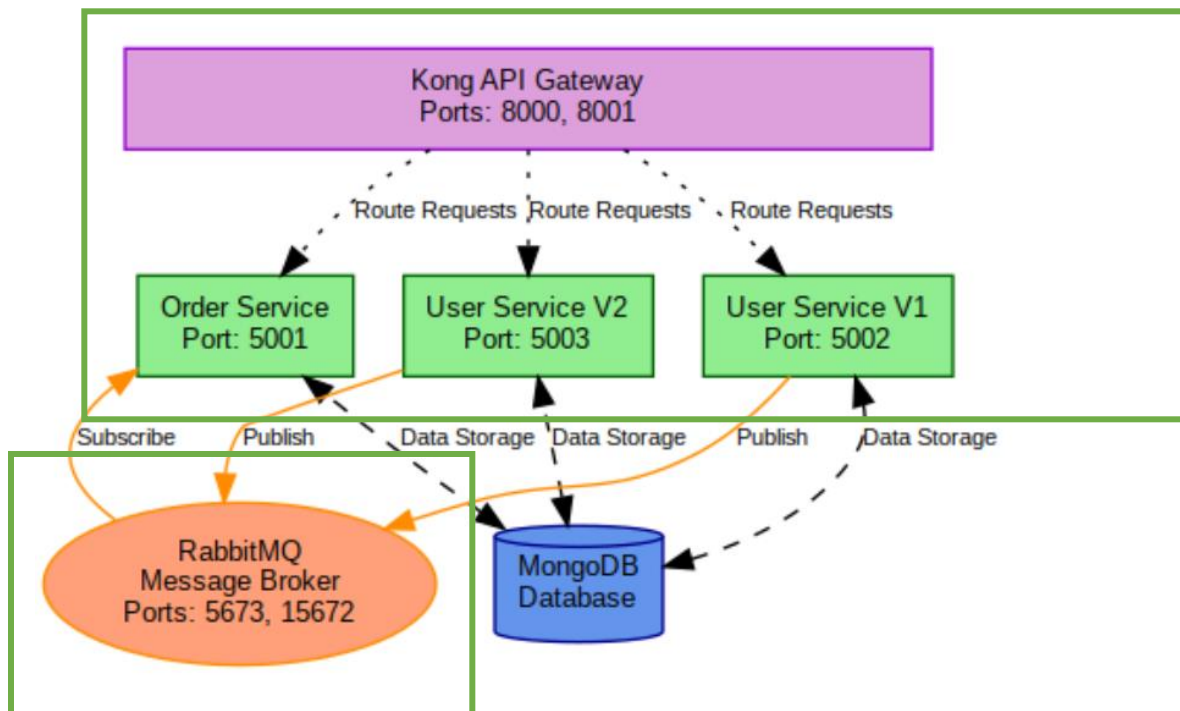
Overview of Integration Testing Scope

- **API Gateway Routing (Kong):** Verify Kong correctly routes requests to the User and Order services based on defined paths (/users/, /orders/) and HTTP methods (POST, PUT, GET). Load balancing between User Service v1 and v2 must be validated based on configurable weights.
- **Microservice API Functionality:** Test the User and Order service APIs for creating (POST), updating (PUT), and retrieving (GET - Order service only) data. Data validation based on JSON schemas (user_schema.json, order_schema.json) must be enforced.
- **Event-Driven Synchronization (RabbitMQ):** Confirm that updates to user information (email, delivery address) in the User service trigger RabbitMQ events, which in turn update the corresponding information in the Order service.
- **Database Interactions (MongoDB):** Verify that both services can successfully read and write data to their respective MongoDB databases, adhering to the defined schemas. This includes ensuring data integrity.

- **Version Management (Strangler Pattern):** Test the API Gateway's routing capability between different versions of the User service (v1 and v2) according to the configured weights.
- **Data Integrity:** Ensure that data stored in the MongoDB databases is consistent and conforms to the JSON schemas (user_schema.json and order_schema.json).

Task 1: Service Deployment and Test Environment Setup(20 Marks)

Check out the repo from [GitHub - TheBarzani/aware-microservices: Scalable and Versionable Microservices Architecture Based on Modern Technologies](https://github.com/TheBarzani/aware-microservices). The API Gateway, and Microservices are installed on one node and RabbitMQ is installed on another node using Docker image [rabbitmq - Official Image | Docker Hub](https://hub.docker.com/_/rabbitmq). The MongoDB is accessible through MongoDB cloud. [MongoDB: The Developer Data Platform | MongoDB](https://www.mongodb.com/docs/atlas/getting-started/). For technical issues of the deployment, please attend your lab, tutorial or PoD sessions to get the support from TAs and PoDs. Document your test environment and deployment in the report.



Task 2: Integration Testing Requirements, Plan, and Test Cases for the Microservices Architecture

Integration Testing Requirements

1. **Validate Inter-Service Communication**
 - Validate REST API endpoints work as expected across services
 - Validate API Gateway routing to the correct microservice. [Hint: set P=0 and P=1 to switch between User service v1 and v2]
 - Validate the Kong API Gateway correctly routes requests based on the strangler pattern (v1 to v2 migration). [Hint: set P= 0.5 to switch between User service v1 and v2]
2. **Validate Event-Driven Synchronization**
 - Validate updates in the **User Microservice** are correctly propagated to the **Order Microservice** via RabbitMQ.
3. **Validate Data Consistency Across Services**
 - Confirm that changes in user details reflect in order details.

Task 2.1. Please number the requirements. The requirements can be further expanded for more fine-grained sub-requirement if any. The requirements cannot be reduced. (0 Marks)

Task 2.2. Define test cases and link the test case to requirements.

Test Case 1: Validate User Creation and Retrieval (5 Marks)

Objective: Ensure that a new user can be created and retrieved via the User Microservice. An example test case definition is provided.

Test ID	TC_01
Description	Create a user and retrieve it via API
Link to Requirement	[Put here the requirement number]
Preconditions	MongoDB is running, API Gateway is configured
Test Steps	1. Send POST /users/ request with valid user data 2. Verify HTTP 201 response 3. Send GET /users/{id} request 4. Validate user details match the original input

Expected Result	The {user="001"} is created and retrieved correctly
Pass/Fail Criteria	User ID in the response matches MongoDB
Test Datasets	[Put here the test datasets]

Test Case 2: Validate Order Creation with Existing User (5 Marks)

Objective: Ensure orders reference an existing user in the database.

Test Case 3: Validate Event-Driven User Update Propagation (5 Marks)

Objective: Ensure that user updates trigger an event and update the linked orders.

Test Case 4: Validate API Gateway Routing (5 Marks)

Objective: Ensure Kong API Gateway routes requests correctly between v1 and v2.

Task 3 : Run integration test for the above four test cases and produce the test results and test report with test coverage of each test requirement. (40 Marks)

Task 4: Write the testing report to cover Task 1-3. (10 Marks)